

L5a: Network-Flow Models

Capacity, conservation, and maximum flow
CHEME 5800 Cornell University

Objectives for Today

How much flow can a capacity-limited network deliver, and how can we show that this amount is the maximum?

Today's concepts

- **Formulate the problem**: define capacities, conservation, and net source outflow.
- **Apply Ford–Fulkerson**: construct residual graphs, revise earlier routes, and choose augmenting paths.
- **Verify maximum flow**: check feasibility and use an upper bound from edge capacities to prove that more flow is impossible.

Lecture: Network-flow models: capacity, conservation, and maximum flow

Lecture and Example

Lecture: Network-flow models

Formulate a flow problem, increase flow through residual paths, and establish a bound on what the network can deliver.

Example: Maximum flow in a worker–task network

How many assignments can the workers complete? Build the network, compute flows using two path-search rules, and check the results.

One unit of flow represents **one completed assignment**.

Formulating a Maximum-Flow Problem

Represent the application as a finite directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with vertices $\mathcal{V}$ and edges $\mathcal{E}$.

- The **source** s supplies flow; the **sink** t receives it, with $s \neq t$.
- Each original edge (u, v) has a finite **capacity** $c(u, v) \geq 0$.
- The **flow** $f(u, v)$ is the amount assigned to that edge, in the same units as the capacity.

Worker availability and task limits become capacities. Data transmission and material transport use the same formulation.

Capacity and Conservation Define Feasibility

Each original edge's flow $f(u, v)$ must stay within its capacity $c(u, v)$:

$$\underbrace{0 \leq f(u, v) \leq c(u, v)}_{\text{capacity constraint}}, \quad (u, v) \in \mathcal{E}.$$

At an intermediate vertex $v \in \mathcal{V} \setminus \{s, t\}$, incoming and outgoing flow must balance:

$$\underbrace{\sum_{u:(u,v) \in \mathcal{E}} f(u, v)}_{\text{inflow}} = \underbrace{\sum_{w:(v,w) \in \mathcal{E}} f(v, w)}_{\text{outflow}}.$$

Intermediate vertices neither create nor consume flow. A **feasible flow** satisfies both constraints.

Maximize Net Flow Leaving the Source

The flow value $|f|$ measures net outflow from the source s :

$$|f| = \underbrace{\sum_{v:(s,v) \in \mathcal{E}} f(s,v)}_{\text{leaving } s} - \underbrace{\sum_{u:(u,s) \in \mathcal{E}} f(u,s)}_{\text{entering } s}.$$

The source and sink are exempt from conservation: the source creates flow and the sink consumes it. Conservation at the intermediate vertices makes $|f|$ equal to the net inflow at the sink t .

The maximum-flow problem seeks a feasible flow f^* with the largest value:

$$|f^*| = \max_{f \text{ feasible}} |f|.$$

Equivalently: imagine an edge entering the source from outside the network and an edge leaving the sink to outside the network, each carrying $|f|$; then every vertex balances.

Ford–Fulkerson Method

Start with zero flow. Repeatedly find an **augmenting path**: a source-to-sink route along which we can still send more flow.

A **residual graph** records where we can add flow and where we can take back flow assigned earlier:

- **Forward edges** record unused capacity.
- **Backward edges** record flow we can cancel to revise an earlier routing choice.

Ford–Fulkerson is a **method**: it leaves the path-selection rule open, and different rules give different algorithms.

What Is a Residual Graph?

Suppose the original edge (u, v) has capacity $c(u, v) = 5$ and currently carries flow $f(u, v) = 3$.

Forward: add flow

Unused capacity from u to v :

$$c(u, v) - f(u, v) = 2.$$

Backward: cancel flow

Flow we can cancel using $v \rightarrow u$:

$$f(u, v) = 3.$$

A backward residual edge is bookkeeping. It reduces an existing flow; it does not require a physical reverse connection.

Defining Residual Capacities

Assume no self-loops or oppositely directed pairs of original edges. For an ordered pair (u, v) , the residual capacity is:

$$\text{residual}(u, v) = \begin{cases} c(u, v) - f(u, v), & (u, v) \in \mathcal{E}, \\ f(v, u), & (v, u) \in \mathcal{E}, \\ 0, & \text{otherwise.} \end{cases}$$

The residual graph $\mathcal{G}_f = (\mathcal{V}, \mathcal{E}_f)$ retains all vertices and includes only edges with positive residual capacity:

$$\mathcal{E}_f = \{(u, v) \in \mathcal{V} \times \mathcal{V} : \text{residual}(u, v) > 0\}.$$

Backward edges: because $\mathcal{E}_f$ is built from all ordered pairs, (v, u) appears whenever $f(u, v) > 0$, even though (v, u) is not an original edge.

How Do We Augment the Flow?

An **augmenting path** P is a directed path from s to t in the residual graph. Every edge on it has positive residual capacity.

The largest increase Δ along the chosen path is its **bottleneck capacity**:

$$\Delta = \min_{(u,v) \in P} \text{residual}(u,v).$$

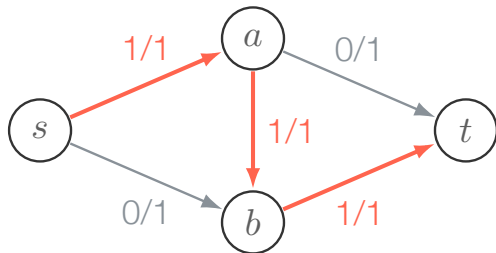
- On a forward edge, Δ limits how much flow we can add.
- On a backward edge, Δ limits how much flow we can cancel.

Use the same Δ on every path edge to preserve conservation.

A Reverse Edge Revises an Earlier Route

All five original edges have capacity one.

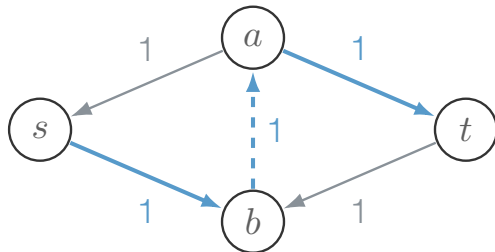
First flow: $|f| = 1$



Labels: flow / capacity

$s \rightarrow a \rightarrow b \rightarrow t$

Residual path: $\Delta = 1$



Labels: residual capacity

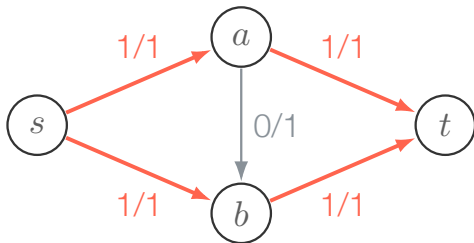
$s \rightarrow b \rightarrow a \rightarrow t$

The dashed step $b \rightarrow a$ cancels the earlier flow on (a, b) .

Rerouting Raises the Flow Value to Two

Augment along $s \rightarrow b \rightarrow a \rightarrow t$ with $\Delta = 1$.

Revised flow: $|f| = 2$



Labels: flow / capacity

$s \rightarrow a \rightarrow t$ and $s \rightarrow b \rightarrow t$

Original edge updates

Edge	Before	After
(s, b)	0	1
(a, b)	1	0
(a, t)	0	1

Other edge flows stay at one; conservation is preserved.

Ford–Fulkerson Procedure

Initialize $f(u, v) = 0$ for every original edge. Then repeat:

1. Construct the residual graph and find a path P from s to t . If no path exists, stop.
2. Find the bottleneck Δ along P .
3. Add Δ to each forward edge's flow; subtract Δ from the original flow corresponding to each backward edge.
4. Recompute residual capacities before searching again.

Each augmentation preserves feasibility and increases $|f|$ by Δ .

Edmonds–Karp Uses Breadth-First Search

Search the residual graph using **breadth-first search**, following only edges with positive residual capacity.

- Choose an augmenting path with the fewest edges.
- Neighbor order breaks ties between equal-length paths.
- Update residual capacities and search again after each augmentation.

With adjacency lists and exact arithmetic, the worst-case running time is:

$$\mathcal{O}(|\mathcal{V}| |\mathcal{E}|^2).$$

The bound depends on graph size, not capacity values.

Reference: [Sedgewick and Wayne, Algorithms: Maximum Flow](#)

Two Path-Selection Rules for the Same Method

The [worker–task example](#) applies the Ford–Fulkerson method with two path-selection rules to the same capacities.

Path-selection rule	Behavior
Depth-first search	Takes the first augmenting path it reaches; may choose longer paths
Breadth-first search (Edmonds–Karp)	Chooses an augmenting path with the fewest edges

Both searches give the same maximum-flow value. Their individual worker–task assignments may differ.

With integer capacities, each augmentation adds at least one unit, so either search terminates.

Verifying Feasibility and Maximum Flow

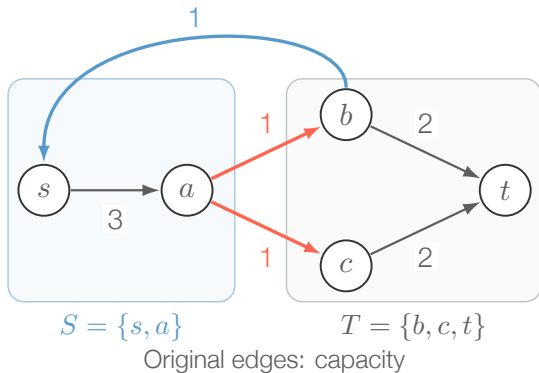
Ford–Fulkerson stops when no augmenting path remains. How do we know the returned flow is the largest possible, and not merely the largest this sequence of paths could find? First, check feasibility:

1. **Capacities**: every original edge satisfies $0 \leq f(u, v) \leq c(u, v)$.
2. **Conservation**: each intermediate vertex has equal total inflow and outflow.
3. **Source–sink balance**: net source outflow equals net sink inflow.

These checks establish only that the flow is allowed. To prove it is maximum, we need an upper bound that the feasible flow reaches.

A Source–Sink Cut Gives an Upper Bound

A **source–sink cut** partitions $\mathcal{V}$ into disjoint sets S and T , with $s \in S$ and $t \in T$.



The cut capacity is the sum of the original edge capacities from S to T :

$$c(S, T) = \sum_{\substack{(u,v) \in \mathcal{E} \\ u \in S, v \in T}} c(u, v).$$

The two **outgoing cut edges** give $c(S, T) = 1 + 1 = 2$.

The **edge $b \rightarrow s$** is an original edge that enters the source. It crosses from T to S and adds no cut capacity.

Cut Balance Bounds Every Feasible Flow

Sum net outflows over vertices in S . Internal edge flows cancel, and conservation leaves only the source's net outflow:

$$|f| = \underbrace{\sum_{\substack{(u,v) \in \mathcal{E} \\ u \in S, v \in T}} f(u,v)}_{S \rightarrow T} - \underbrace{\sum_{\substack{(u,v) \in \mathcal{E} \\ u \in T, v \in S}} f(u,v)}_{T \rightarrow S}.$$

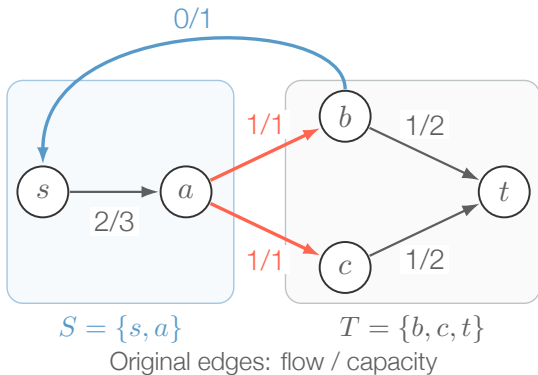
Forward flow is at most its capacity; return flow is nonnegative. Thus:

$$|f| \leq \sum_{\substack{(u,v) \in \mathcal{E} \\ u \in S, v \in T}} c(u,v) = c(S,T).$$

Every source–sink cut therefore bounds every feasible flow.

No Augmenting Path Identifies a Cut

When t is unreachable, let S contain the vertices reachable from s in the residual graph; $T = \mathcal{V} \setminus S$.



$S \rightarrow T$: **saturated**

Unused capacity would let the residual search enter T .

$T \rightarrow S$: **zero flow**

Positive flow would create a backward residual edge into T .

$S \rightarrow T$ edges are full and the $T \rightarrow S$ edge is empty, so $|f| = c(S, T) = 2$.

The Stopping Condition Proves Maximum Flow

Substitute saturated $S \rightarrow T$ edges and zero $T \rightarrow S$ flows into the cut balance:

$$|f| = c(S, T).$$

The feasible flow reaches a bound on every feasible flow. It is therefore maximum, and the cut has minimum capacity.

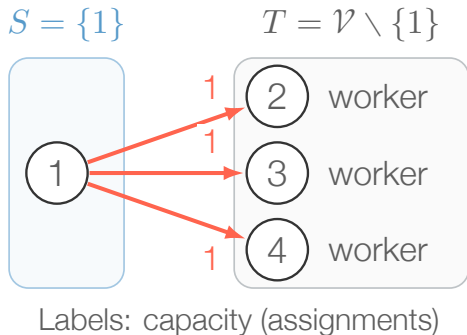
The **maximum-flow/minimum-cut theorem** states:

$$\max_{f \text{ feasible}} |f| = \min_{(S, T) \text{ source-sink cut}} c(S, T).$$

Reference: [David Karger, MIT 6.854: Flow](#)

Three Assignments Reach the Source-Cut Bound

The default worker-task network has three source-to-worker edges, each with capacity one assignment.



Put the source, node 1, in S and every other vertex in T . The cut capacity is:

$$c(S, T) = 1 + 1 + 1 = 3.$$

The [worked example](#) attains this bound:

$$|f| = c(S, T) = 3.$$

Therefore, the maximum is **three assignments**. Each worker can take one assignment, so the fourth task goes unassigned.

Next: [L5b changes capacities](#) and examines how the maximum number of assignments changes.

Summary

We formulated, computed, and verified maximum flow.

Key takeaways

- **Capacity and conservation.** We maximized net source outflow subject to edge capacities and intermediate flow balances.
- **Residual augmentation.** We used bottlenecks to add or cancel flow while preserving feasibility, and compared depth-first search with Edmonds–Karp, which chooses paths with the fewest edges.
- **Proof of maximum flow.** We matched feasible flow to a cut bound, proving a maximum of three assignments in our example.

In [L5b](#), we examine how capacity changes affect this maximum.